



Programación funcional

Fundamentos de Lenguajes de Programación – IIC2560

Yadran Eterovic S. (yadran@uc.cl)

Todo lo estudiado hasta aquí es válido para lenguajes de programación que basan su modelo computacional en la transformación del *estado*

El concepto fundamental es el de *variable modificable*:

- un contenedor que tiene un nombre y al cual durante la computación se le puede asignar diversos valores mientras la asociación (entre nombre y contenedor) se mantiene en el *ámbito*

Todo lo estudiado hasta aquí es válido para lenguajes de programación que basan su modelo computacional en la transformación del *estado*

Las variables del programa

Lenguajes *imperativos*

El concepto fundamental es el de *variable modificable*:

- un contenedor que tiene un nombre y al cual durante la computación se le puede asignar diversos valores mientras la asociación (entre nombre y contenedor) se mantiene en el *ámbito*

Modelo computacional
imperativo

Base común: La arquitectura von Neumann

Todo lo estudiado hasta aquí es válido para lenguajes de programación que basan su modelo computacional en la transformación del *estado*

Las variables del programa

El concepto fundamental es el de *variable modificable*:

- un contenedor que tiene un nombre y al cual durante la computación se le puede asignar diversos valores mientras la asociación (entre nombre y contenedor) se mantiene en el *ámbito*

El principal constructo es la *asignación*:

- cambia el valor de una variable (el *r-value*)
... pero no la asociación entre el nombre y (la ubicación del) el contenedor (el *l-value*)

Lenguajes *imperativos*

Modelo computacional *imperativo*

Base común: La arquitectura von Neumann

Conjunto de asociaciones entre nombres y objetos denotables que existen durante la ejecución

... en un punto específico del programa

... y en un momento específico de la computación

No existe al nivel del computador físico

En lenguajes funcionales, no hay estado ni variables modificables

- no hay necesidad de la asignación
- la iteración se vuelve menos importante

Un *loop* modifica repetidamente el estado

La computación ocurre mediante la *reescritura* de expresiones

- cambios que ocurren sólo dentro el ámbito y no involucran una memoria

El manejo de **funciones de orden superior** es fundamental

La **recursión** es el principal constructo de control de secuencia

Mapping

Función (en matemáticas):

- *correspondencia* de miembros de un conjunto —el **dominio**— a otro conjunto —el **recorrido**

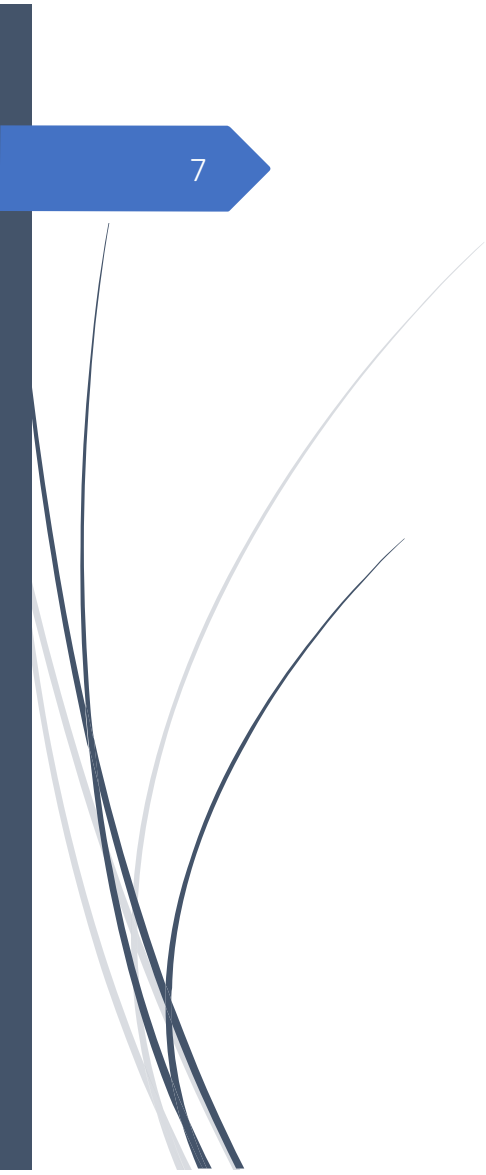
La definición de una función especifica el dominio, el recorrido y la correspondencia

Una función es aplicada a un elemento del dominio —el parámetro de la función

El dominio puede ser el producto cruz de varios conjuntos —más de un parámetro

Una función produce un elemento del recorrido

Descrita mediante una expresión (o una tabla)



El orden de evaluación de las expresiones es controlado por recursión y expresiones condicionales

No producen efectos laterales y no dependen de valores externos

→ siempre hacen corresponder un elemento particular del dominio al mismo elemento del rango

El orden de evaluación de las expresiones es controlado por recursión y expresiones condicionales

No producen efectos laterales y no dependen de valores externos

→ siempre hacen corresponder un elemento particular del dominio al mismo elemento del rango

... en lugar de secuenciación e iteración

No así los subprogramas en lenguajes imperativos

Definición de funciones

$\text{cube}(x) \equiv x * x * x$, en que x es un número real

Definición de funciones

"se define como"

 $\text{cube}(x) \equiv x * x * x$, en que x es un número real

Nombre

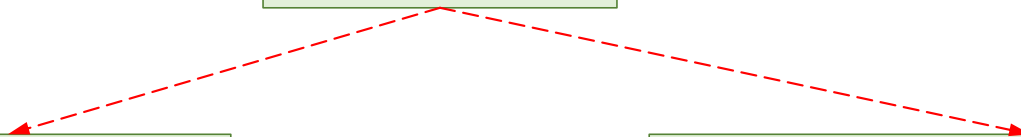
Lista de parámetros
(en este caso, sólo x)Expresión que especi-
fica la correspondenciaEspecificación del
dominio (y recorrido)

x puede representar cualquier elemento del dominio
... pero queda fijo y representa un elemento específico
durante la evaluación de la expresión

Pueden separarse ...

La definición de la función

La acción de asignarle un nombre a la función



Pueden separarse ...

La definición de la función

La acción de asignarle un nombre a la función

La notación *lambda* [A.Church, 1941] ofrece un método para definir funciones *anónimas*:

- una **expresión lambda** especifica los parámetros y la correspondencia de una función

Puede tener más de un parámetro

$\lambda(x) x^* x^* x$

Expresión lambda *aplicada* al parámetro 2

$(\lambda(x) x^* x^* x)(2)$

... y que resulta en el valor 8

Forma funcional (o *función de orden superior*)

Función que recibe una o más funciones como parámetros
... o que produce una función como resultado
... o ambas

Forma funcional (o *función de orden superior*)

Función que recibe una o más funciones como parámetros
... o que produce una función como resultado
... o ambas

Composición de funciones:

- dos parámetros funcionales
- produce una función cuyo valor es el primer parámetro funcional real (o argumento funcional) aplicado al resultado del segundo

• e.g., si

$$\dots f(x) \equiv x + 2$$

$$\dots g(x) \equiv 3 * x$$

$$\dots h \equiv f \circ g$$

$$\rightarrow h(x) \equiv f(g(x)) \equiv (3 * x) + 2$$

Forma funcional (o *función de orden superior*)

Función que recibe una o más funciones como parámetros
... o que produce una función como resultado
... o ambas

Composición de funciones:

- dos parámetros funcionales
- produce una función cuyo valor es el primer parámetro funcional real (o argumento funcional) aplicado al resultado del segundo
- e.g., si
... $f(x) \equiv x + 2$
... $g(x) \equiv 3 * x$
... $h \equiv f \circ g$
→ $h(x) \equiv f(g(x)) \equiv (3 * x) + 2$

Aplicar a todos (o función *map*), denotada por α

- un parámetro funcional
- si se aplica a una lista de parámetros, aplica el parámetro funcional a cada valor en la lista y recolecta los resultados en una lista o secuencia
- e.g., si
... $h(x) \equiv x * x$
→ $\alpha(h(2, 3, 4))$ produce (4, 9, 16)

Lenguajes funcionales

→ imitar las funciones matemáticas

Un programa:

- definiciones de funciones
- especificaciones de aplicaciones de funciones

La ejecución de un programa:

- evaluación de las aplicaciones de funciones

La ejecución de una función siempre produce el mismo resultado cuando se le pasan los mismos parámetros → **transparencia**

referencial:

- la semántica es más simple que la de los lenguajes imperativos
- el *testing* es más fácil

Componentes de un lenguaje funcional

Un conjunto de funciones primitivas

Un conjunto de formas funcionales

Una operación de aplicación de función

Una o más estructuras para representar datos

Componentes de un lenguaje funcional

Un conjunto de funciones primitivas

Un conjunto de formas funcionales

Una operación de aplicación de función

Una o más estructuras para representar datos

Para construir funciones complejas
a partir de las funciones primitivas

Para representar los parámetros y los
valores calculados por las funciones

Scheme

[G.J. Sussman & G.L. Steele, 1975 (MIT)]

Lenguaje pequeño

Alcance estático

Funciones como entidades de primera clase

Scheme

[G.J. Sussman & G.L. Steele, 1975 (MIT)]

Lenguaje pequeño

Alcance estático

Funciones como entidades de primera clase

- Sin tipos de datos
- Sintaxis y semántica simples
- Pueden ser valores de expresiones, elementos de listas, parámetros, resultados de funciones

Scheme

[G.J. Sussman & G.L. Steele, 1975 (MIT)]

Lenguaje pequeño

Alcance estático

Funciones como entidades de primera clase

- Sin tipos de datos
- Sintaxis y semántica simples

- Pueden ser valores de expresiones, elementos de listas, parámetros, resultados de funciones

El intérprete en modo interactivo es un *loop* infinito *read-evaluate-print* → repetidamente:

- lee una expresión tipeada por el usuario
- la interpreta
- muestra en pantalla el valor resultante

Interpretación usada
también en Ruby y Python

Scheme

[G.J. Sussman & G.L. Steele, 1975 (MIT)]

Lenguaje pequeño

Alcance estático

Funciones como entidades de primera clase

- Sin tipos de datos
- Sintaxis y semántica simples

- Pueden ser valores de expresiones, elementos de listas, parámetros, resultados de funciones

El intérprete en modo interactivo es un *loop* infinito *read-evaluate-print* → repetidamente:

- lee una expresión tipeada por el usuario
- la interpreta
- muestra en pantalla el valor resultante

Interpretación usada
también en Ruby y Python

Las expresiones son interpretadas por la función **EVAL**:

- literales evalúan a sí mismos
 - llamadas a funciones primitivas ...
- ... primero, se evalúa cada uno de los parámetros
- ... luego, la función es aplicada a los valores resultantes

Definición de funciones

Funciones numéricas primitivas

<i>Expression</i>	<i>Value</i>
42	42
(* 3 7)	21
(+ 5 7 8)	20
(- 5 6)	-1
(- 15 7 2)	6
(- 24 (* 4 3))	12

Además,
MODULO, ROUND, MAX, MIN, LOG, SIN, SQRT

Definición de funciones

Funciones numéricas primitivas

<i>Expression</i>	<i>Value</i>
42	42
(* 3 7)	21
(+ 5 7 8)	20
(- 5 6)	-1
(- 15 7 2)	6
(- 24 (* 4 3))	12

Además,
MODULO, ROUND, MAX, MIN, LOG, SIN, SQRT

Funciones predicado numéricas

<i>Function</i>	<i>Meaning</i>
=	Equal
<>	Not equal
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
EVEN?	Is it an even number?
ODD?	Is it an odd number?
ZERO?	Is it zero?

Definición de funciones

Expresión lambda
y una aplicación
(que produce 49)

```
(LAMBDA (x) (* x x))
```

```
((LAMBDA (x) (* x x)) 7)
```

Una expresión lambda
puede tener cualquier
número de parámetros

```
(LAMBDA (a b c x) (+ (* a x x) (* b x) c))
```

Definición de funciones (cont.)

La función **DEFINE** es una *forma especial* (interpretada por **EVAL** de manera diferente)

```
(DEFINE symbol expression)
```

```
(DEFINE pi 3.14159)  
(DEFINE two_pi (* 2 pi))
```

Vincula un nombre al **valor** de una expresión

Análogo a la declaración de una constante en un lenguaje imperativo

Definición de funciones (cont.)

27

La función **DEFINE** es una forma especial (interpretada por **EVAL** de manera diferente)

```
(DEFINE symbol expression)
```

```
(DEFINE pi 3.14159)
(DEFINE two_pi (* 2 pi))
```

Vincula un nombre al valor de una expresión

Análogo a la declaración de una constante en un lenguaje imperativo

```
(DEFINE (function_name parameters)
  (expression)
)
```

```
(DEFINE (square number) (* number number))
```

```
(DEFINE (hypotenuse side1 side2)
  (SQRT(+ (square side1) (square side2)))
)
```

Vincula una expresión **LAMBDA** a un nombre, sin usar la palabra "**LAMBDA**"

Definición de funciones (cont.)

28

Tres constructos para flujo de control:

- IF
- ...

(IF predicate then_expression else_expression)

```
(DEFINE (factorial n)
  (IF (<= n 1)
      1
      (* n (factorial (- n 1)))))
```

Definición de funciones (cont.)

29

Tres constructos para flujo de control:

- IF
- COND
- ...

(IF predicate then_expression else_expression)

```
(DEFINE (factorial n)
  (IF (<= n 1)
    1
    (* n (factorial (- n 1)))))
```

```
(COND
  (predicate1 expression1)
  (predicate2 expression2)
  ...
  (predicaten expressionn)
  [ (ELSE expressionn+1) ]
)
```

```
(DEFINE (leap? year)
  (COND
    ((ZERO? (MODULO year 400)) #T)
    ((ZERO? (MODULO year 100)) #F)
    (ELSE (ZERO? (MODULO year 4)))
  ))
```

Definición de funciones (cont.)

30

Tres constructos para flujo de control:

- IF
- COND
- recursión

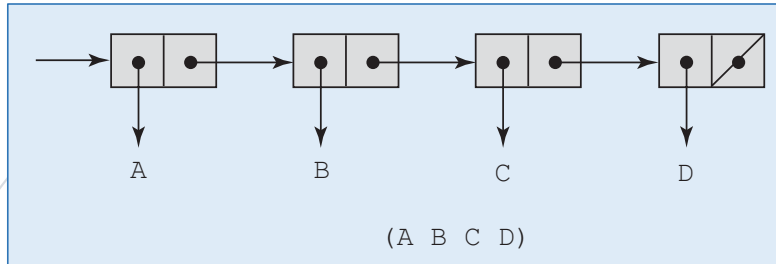
(IF predicate then_expression else_expression)

```
(DEFINE (factorial n)
  (IF (<= n 1)
    1
    (* n (factorial (- n 1)))))
```

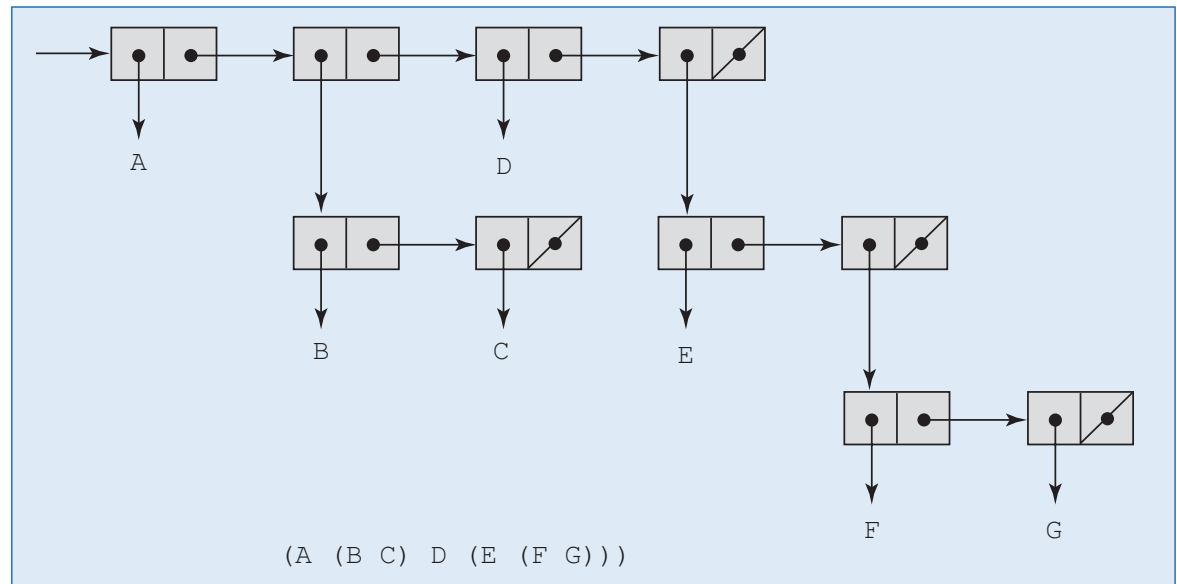
```
(COND
  (predicate1 expression1)
  (predicate2 expression2)
  ...
  (predicaten expressionn)
  [ (ELSE expressionn+1) ]
)
```

```
(DEFINE (leap? year)
  (COND
    ((ZERO? (MODULO year 400)) #T)
    ((ZERO? (MODULO year 100)) #F)
    (ELSE (ZERO? (MODULO year 4)))
  ))
```

Definición de funciones (cont.)



Listas



Definición de funciones (cont.)

(QUOTE A) **returns** A
 (QUOTE (A B C)) **returns** (A B C)

(QUOTE (A B)) se puede escribir
 simplifcadamente como '(A B)

(CAR '(A B C)) **returns** A
 (CAR '((A B) C D)) **returns** (A B)
 (CAR 'A) **is an error because A is not a list**
 (CAR '(A)) **returns** A
 (CAR '()) **is an error**
 (CDR '(A B C)) **returns** (B C)
 (CDR '((A B) C D)) **returns** (C D)
 (CDR 'A) **is an error**
 (CDR '(A)) **returns** ()
 (CDR '()) **is an error**

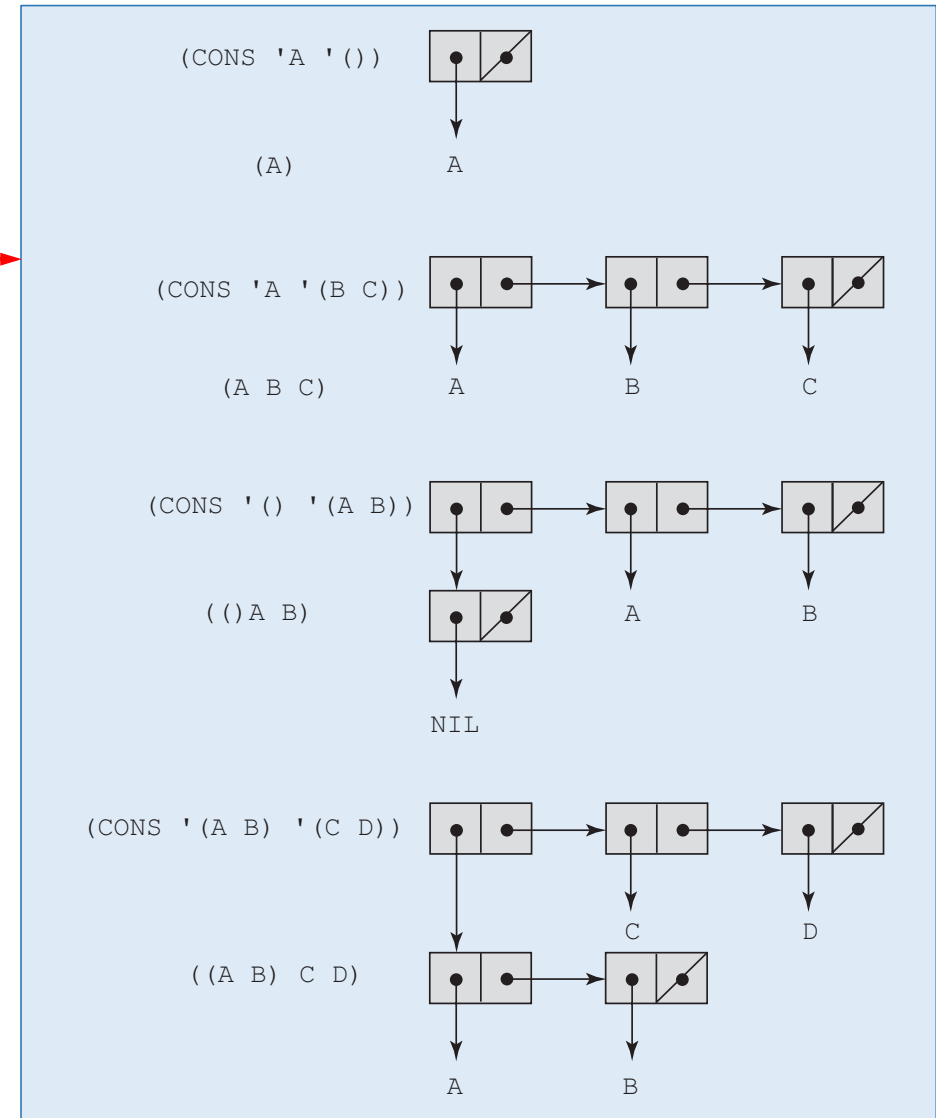
(DEFINE (second a_list) (CAR (CDR a_list)))

(second '(A B C))

... produce **B**

Definición de funciones (cont.)

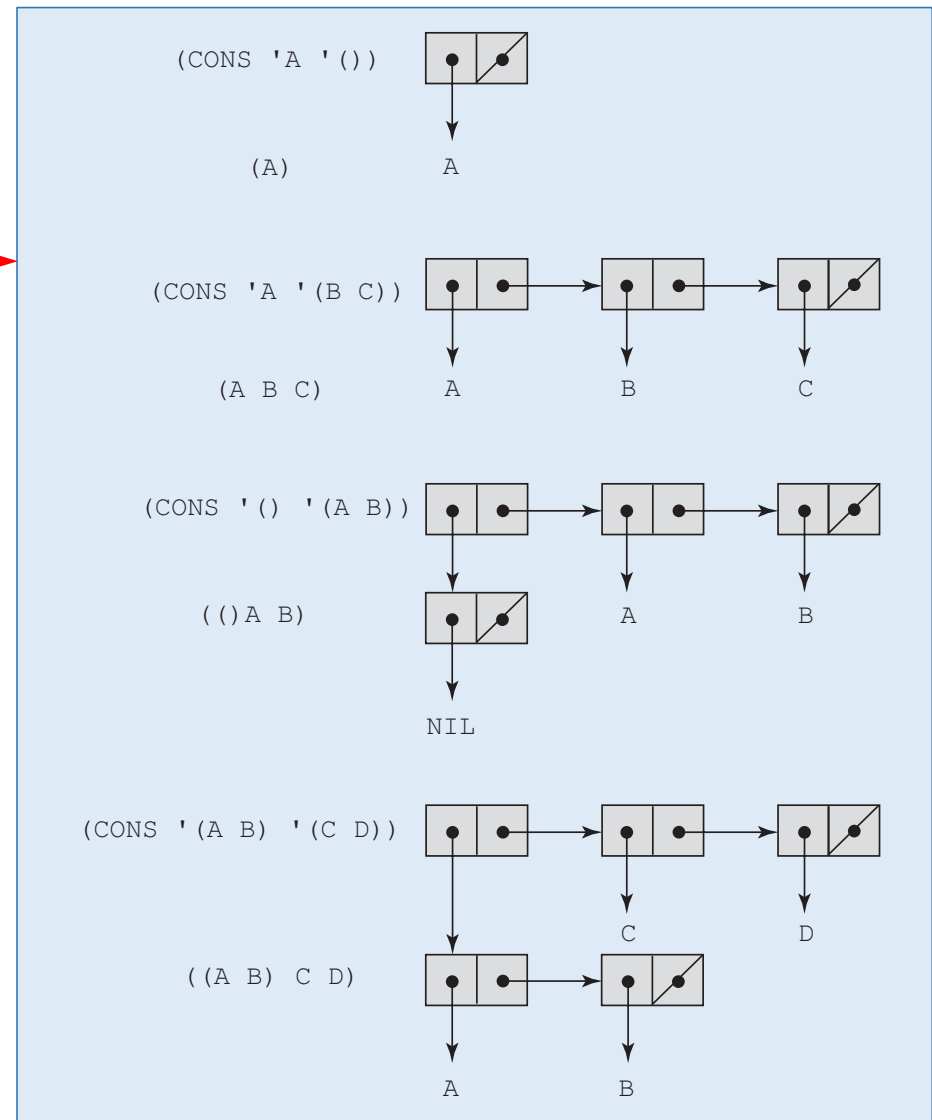
(CONS 'A '()) returns (A)
 (CONS 'A '(B C)) returns (A B C)
 (CONS '() '(A B)) returns (() A B)
 (CONS '(A B) '(C D)) returns ((A B) C D)



Definición de funciones (cont.)

(CONS 'A '()) returns (A)
 (CONS 'A '(B C)) returns (A B C)
 (CONS '() '(A B)) returns (() A B)
 (CONS '(A B) '(C D)) returns ((A B) C D)

(CONS 'A 'B) = (A . B)



Definición de funciones (cont.)

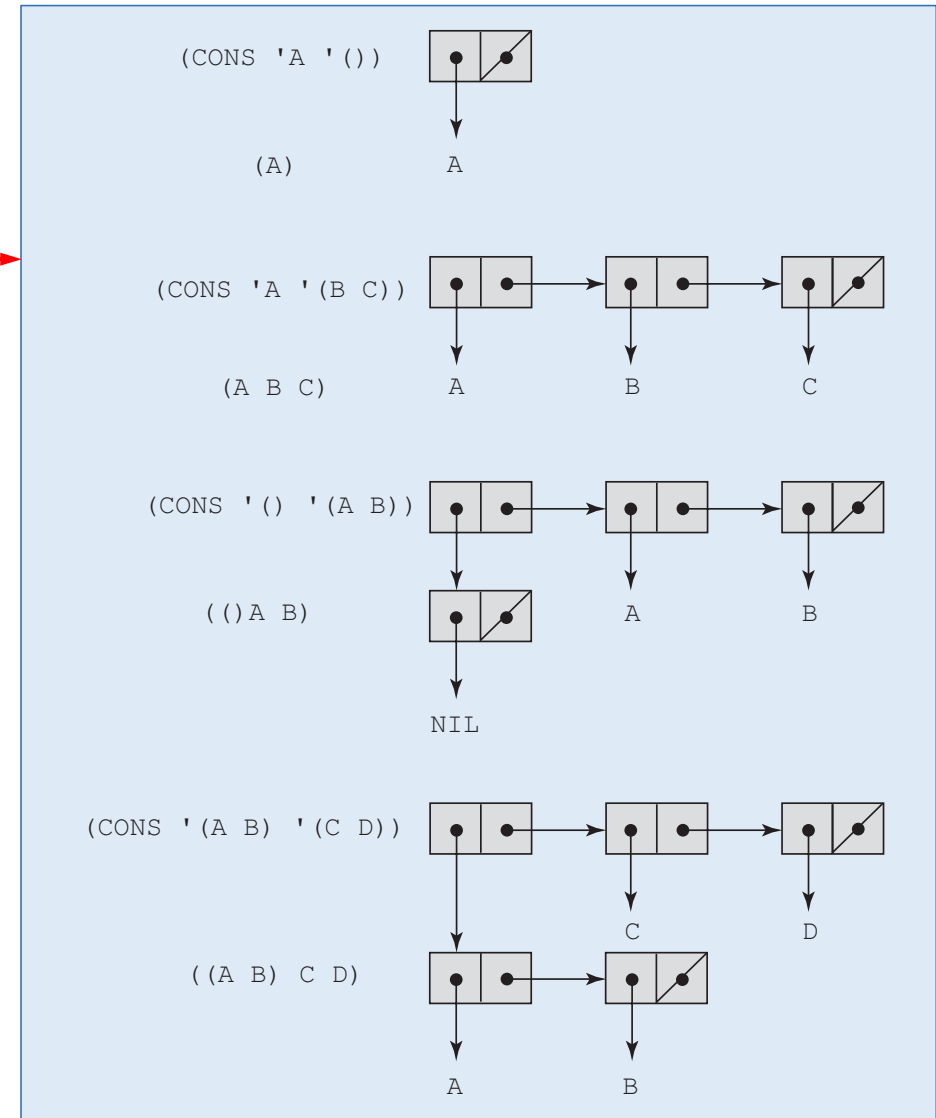
`(CONS 'A '())` returns `(A)`
`(CONS 'A '(B C))` returns `(A B C)`
`(CONS '() '(A B))` returns `(() A B)`
`(CONS '(A B) '(C D))` returns `((A B) C D)`

`(CONS 'A 'B) = (A . B)`

`(LIST 'apple 'orange 'grape)`

`(CONS 'apple (CONS 'orange (CONS 'grape '())))`

`(apple orange grape)`



(EQ? 'A 'A) returns #T
(EQ? 'A 'B) returns #F
(EQ? 'A '(A B)) returns #F
(EQ? '(A B) '(A B)) returns #F or #T
(EQ? 3.4 (+ 3 0.4)) returns #F or #T

(LIST? '(X Y)) returns #T
(LIST? 'X) returns #F
(LIST? '()) returns #T

(EQV? 'A 'A) returns #T
(EQV? 'A 'B) returns #F
(EQV? 3 3) returns #T
(EQV? 'A 3) returns #F
(EQV? 3.4 (+ 3 0.4)) returns #T
(EQV? 3.0 3) returns #F

(NULL? '(A B)) returns #F
(NULL? '()) returns #T
(NULL? 'A) returns #F
(NULL? '(())) returns #F

```
(member 'B ' (A B C)) returns #T  
(member 'B ' (A C D E)) returns #F
```

(member 'B '(A B C)) **returns** #T
(member 'B '(A C D E)) **returns** #F

```
(DEFINE (member atm a_list)
  (COND
    ((NULL? a_list) #F)
    ((EQ? atm (CAR a_list)) #T)
    (ELSE (member atm (CDR a_list)))
  ))
```

```
(DEFINE (equalsimp list1 list2)
  (COND
    ((NULL? list1) (NULL? list2))
    ((NULL? list2) #F)
    ((EQ? (CAR list1) (CAR list2))
      (equalsimp (CDR list1) (CDR list2)))
    (ELSE #F)
  ))
```

```
(DEFINE (equalsimp list1 list2)
  (COND
    ((NULL? list1) (NULL? list2))
    ((NULL? list2) #F)
    ((EQ? (CAR list1) (CAR list2))
      (equalsimp (CDR list1) (CDR list2)))
    (ELSE #F)
  ))
```

```
(append '(A B) '(C D R)) returns (A B C D R)
(append '((A B) C) '(D (E F))) returns ((A B) C D (E F))
```

```
(DEFINE (equal list1 list2)
  (COND
    ((NOT (LIST? list1)) (EQ? list1 list2))
    ((NOT (LIST? list2)) #F)
    ((NULL? list1) (NULL? list2))
    ((NULL? list2) #F)
    ((equal (CAR list1) (CAR list2))
      (equal (CDR list1) (CDR list2)))
    (ELSE #F)
  ))
```

```
(DEFINE (append list1 list2)
  (COND
    ((NULL? list1) list2)
    (ELSE (CONS (CAR list1) (append (CDR list1) list2)))
  ))
```



```
(append '(A B) '(C D R)) returns (A B C D R)  
(append '((A B) C) '(D (E F))) returns ((A B) C D (E F))
```

(append '(A B) '(C D R)) **returns** (A B C D R)
(append '((A B) C) '(D (E F))) **returns** ((A B) C D (E F))

```
(DEFINE (append list1 list2)
  (COND
    ((NULL? list1) list2)
    (ELSE (CONS (CAR list1) (append (CDR list1) list2)))
  ))
```